



ConfigMaps, Secrets, and Volumes: Managing Configuration and Storage for AI Workloads on Kubernetes

A comprehensive guide for DevOps and ML engineers deploying AI workloads at scale

Why Config & Storage Matter in AI

AI applications have unique requirements that make proper configuration and storage management critical:

Configuration Complexity

ML workloads require numerous hyperparameters, paths, and environment settings that change between development and production

Security Requirements

AI systems often need access to sensitive data, APIs, and credentials that must be securely managed

Data Persistence

Large datasets, model weights, and training checkpoints must persist beyond container lifecycles



What Is a ConfigMap?

ConfigMaps are Kubernetes objects that store **non-sensitive configuration data** as key-value pairs:

- Decouple configuration from container images
- Inject settings as environment variables or files
- Update configurations without rebuilding containers
- Share common settings across multiple pods

For AI workloads, ConfigMaps typically store:

- Batch sizes and learning rates
- Model paths and feature toggles
- Logging levels and monitoring settings

Real-world example

A recommendation engine might use ConfigMaps to adjust inference batch sizes during high-traffic periods without redeploying containers.



ConfigMap Example

Creating and using a ConfigMap for an AI application:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ai-config
data:
  BATCH_SIZE: "64"
  MODEL_PATH: "/models/resnet.pt"
  LOG_LEVEL: "INFO"
  FEATURE_GATE_ADVANCED_METRICS: "true"
```

Consuming the ConfigMap in a Pod:

```
containers:
- name: inference-service
  image: ai-company/inference:v1.2
  envFrom:
  - configMapRef:
    name: ai-config
```

This pattern enables updating configuration without touching application code or rebuilding containers — crucial for ML experimentation cycles.

You can also mount ConfigMaps as files, which is ideal for configuration that requires specific file formats, like JSON for model metadata.



What Is a Secret?

Secure Storage

Secrets are [encrypted key-value stores](#) for sensitive information like passwords, tokens, and keys

Security Features

Base64 encoded, encrypted at rest (with proper etcd configuration), and only distributed to nodes that need them

Flexible Consumption

Mount as environment variables or as files within volumes, keeping credentials out of container images

AI-Specific Use Cases

API keys for model repositories (Hugging Face, NVIDIA NGC), database credentials, cloud storage access tokens

❌ **Important:** Base64 encoding is NOT encryption! Secrets are only encrypted at rest if you've configured etcd encryption. Treat Secret YAML files with care.

Secret Example

Creating a Secret for an AI application:

```
apiVersion: v1
kind: Secret
metadata:
  name: ai-secret
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQ= # "password" base64
  HF_API_KEY: aGZfYWJjMTIz # "hf_abc123" base64
  REDIS_AUTH: cmVkaXNwYXNz # "redispass" base64
```

Consuming the Secret in a Pod:

```
containers:
- name: training-job
  image: ai-company/training:v2.1
  env:
  - name: HF_API_KEY
    valueFrom:
      secretKeyRef:
        name: ai-secret
        key: HF_API_KEY
```

Secret Management Options

- Native Kubernetes secrets
- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault

More advanced solutions provide automatic rotation, better encryption, and tighter access controls.

What Is a Volume?



Persistent Storage

Unlike ephemeral container storage, volumes persist beyond pod lifecycles, crucial for stateful AI applications



Storage Types

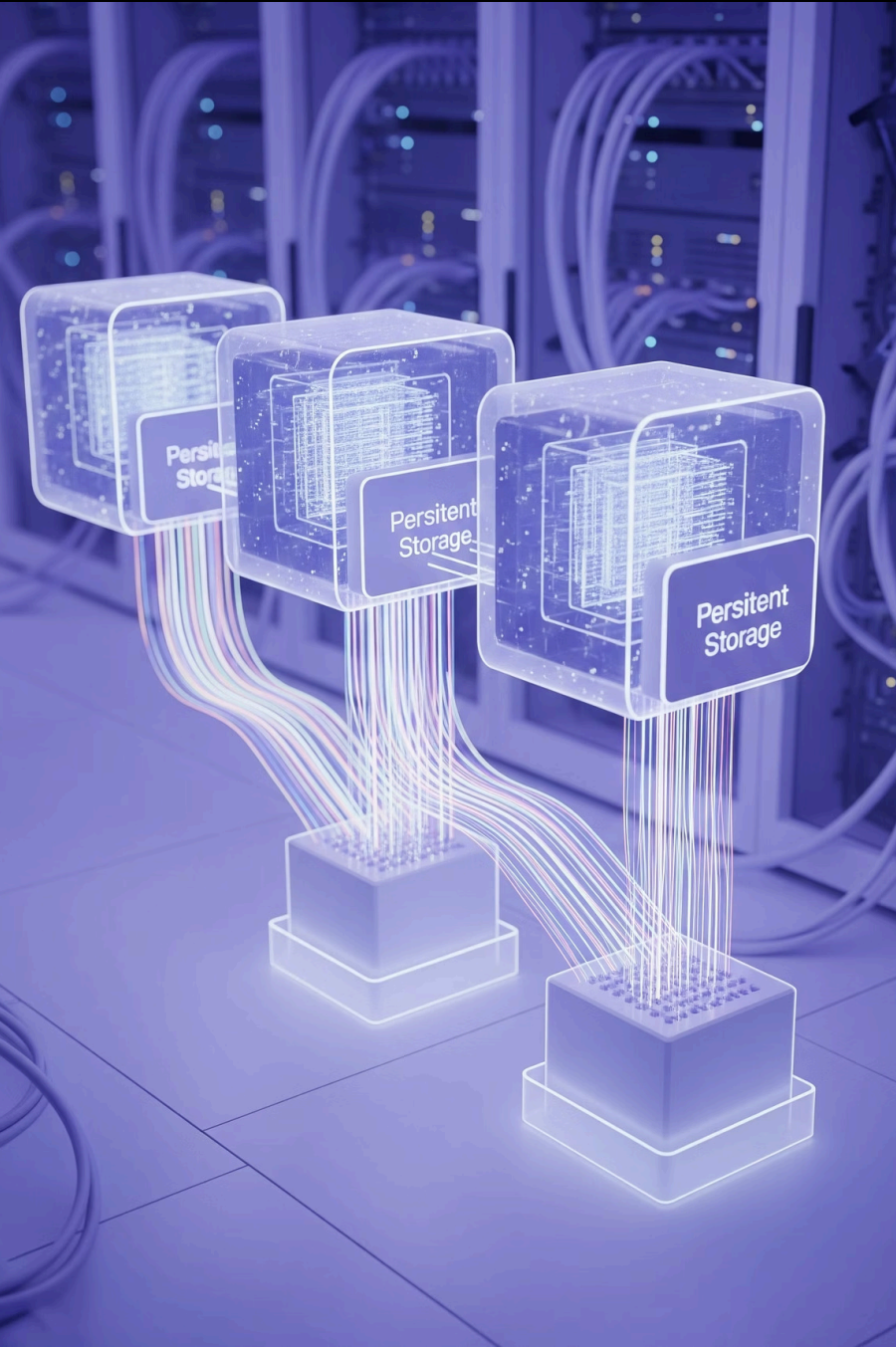
Backed by cloud block storage (EBS, GCP Persistent Disk), networked filesystems (NFS), or specialized AI storage solutions



Access Modes

ReadWriteOnce (RWO), ReadOnlyMany (ROX), or ReadWriteMany (RWX) depending on workload needs

For AI/ML pipelines, volumes are essential for storing datasets, model weights, checkpoints, and experiment artifacts that must persist between training runs.



Volume Example

Creating a PersistentVolumeClaim for model storage:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-model
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 100Gi
  storageClassName: ssd-storage
```

Using the volume in an AI pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: ai-pod
spec:
  containers:
    - name: ai-app
      image: pytorch/pytorch:latest
      volumeMounts:
        - mountPath: /models
          name: model-store
  volumes:
    - name: model-store
      persistentVolumeClaim:
        claimName: pvc-model
```

This setup allows model weights to persist when pods restart, critical for maintaining state across training phases or serving consistent inference results.

How AI Apps Use These Together



1 Data Preparation

Use Volumes to store datasets, ConfigMaps for preprocessing parameters, and Secrets for data source credentials

2 Model Training

ConfigMaps define hyperparameters, Volumes store checkpoints, and Secrets provide access to private model repositories

3 Inference Serving

Volumes hold optimized models, ConfigMaps control batch sizes and timeouts, while Secrets manage API authentication

These components enable reproducible, secure, and scalable multi-stage AI workflows on Kubernetes without sacrificing flexibility or introducing security risks.

Best Practices



Security

- Never hardcode credentials in images or ConfigMaps
- Rotate Secrets regularly, especially in production
- Use RBAC to limit access to sensitive resources
- Consider external secret management tools for production



Organization

- Use namespaces to separate development/staging/production configs
- Label resources clearly for tracking and auditing
- Document ConfigMap options for ML engineers
- Version control your config (not secrets) with code



Performance

- Use SSD-backed storage for model serving
- Consider ReadWriteMany volumes for shared datasets
- Be aware of volume mount performance impacts
- Use cloud-managed storage (S3, GCS) for very large datasets

Following these practices helps avoid common pitfalls when scaling AI workloads in production Kubernetes environments.

Key Takeaways

The Three Pillars

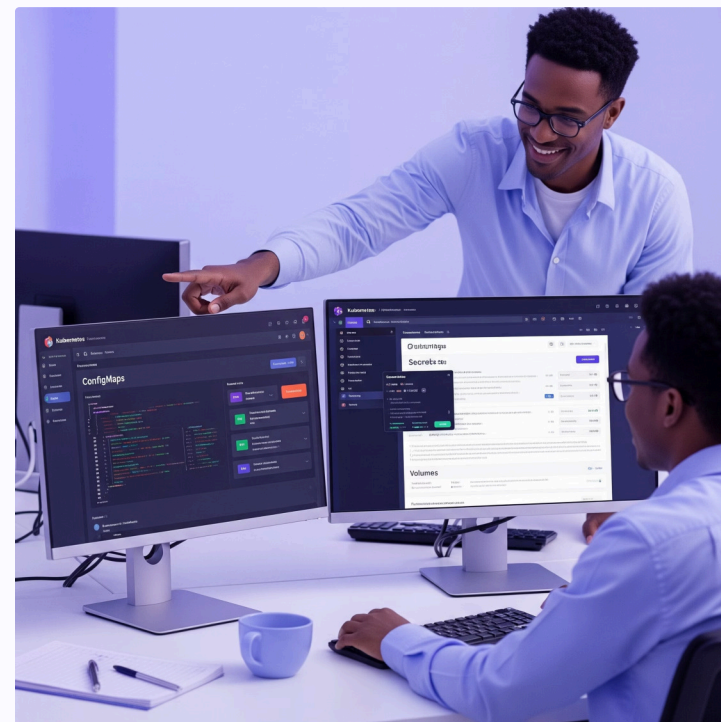
ConfigMaps for configuration, **Secrets** for credentials, **Volumes** for data persistence

Benefits

Increased security, improved flexibility, better reproducibility, and simplified management of complex AI workloads

Next Steps

Implement GitOps workflows, explore external secret management, and investigate specialized ML storage options



These Kubernetes primitives are foundational skills for any engineer working with AI infrastructure at scale.